Pearls AQI Predictor: An End-to-End Serverless Machine Learning System for Air Quality Forecasting in Karachi

Final Project Report (Milestone 8)

Ayesha Salahuddin
Pearls AQI Predictor Project
Karachi, Pakistan

***Abstract*—This report documents the complete design and implementation of the Pearls AQI Predictor, an end-to-end, fully serverless machine learning system that forecasts the Air Quality Index (AQI) of Karachi for the next three days. The system collects live and historical air quality and weather data, engineers features and stores them in a managed feature store, trains and evaluates multiple forecasting models, automates the entire data and training workflow through a CI/CD platform, and serves predictions through a publicly deployed interactive dashboard with hazardous-air alerts. The project integrates the Hopsworks Feature Store and Model Registry, GitHub Actions for automation, and Streamlit for the user interface. Three models, Ridge Regression, Random Forest, and an LSTM neural network, were compared using RMSE, MAE, and R-squared; Random Forest performed best. Model behaviour was explained using SHAP. This report also discusses the engineering decisions, challenges, and honest limitations encountered, including data-source constraints and the inherent difficulty of multi-day air quality forecasting.**

***Index Terms*—air quality index, machine learning, MLOps, feature store, serverless, forecasting, Hopsworks, Streamlit, CI/CD, SHAP**

I. Introduction

Air pollution is a persistent public-health concern in Karachi, where the Air Quality Index frequently reaches unhealthy levels. The objective of the Pearls AQI Predictor is to build a complete, automated system that forecasts the AQI three days into the future using a 100% serverless technology stack. The system was developed in eight milestones spanning environment setup, data collection, exploratory analysis, model training, automation, deployment, and documentation.

The guiding design principle is that the system should run end-to-end without manual intervention: data is collected automatically, the model is retrained on a schedule, and predictions are served continuously through a public web application. This report consolidates the work of all milestones into a single account of the system's architecture, implementation, results, and limitations.

II. System Architecture

The system follows a standard machine-learning operations (MLOps) architecture composed of four stages connected through a central feature store and model registry:

- **Feature Pipeline:** fetches raw weather and pollutant data from external APIs, computes features, and writes them to the Feature Store.
- **Training Pipeline:** reads historical features, trains and evaluates models, and registers the best model in the Model Registry.
- **Automation Layer:** schedules the feature and training pipelines using GitHub Actions.
- **Web Application:** loads the model and features and serves three-day forecasts through an interactive dashboard.

The Hopsworks platform provides both the Feature Store and the Model Registry, acting as the central hub that decouples the pipelines from one another. This separation allows each component to run independently and on its own schedule.

III. Data Collection and Feature Engineering

The system targets Karachi and operates at an hourly resolution. Two categories of data are used: meteorological variables (temperature, humidity, wind speed, pressure) and pollutant concentrations (PM2.5, PM10, NO_2, O_3, SO_2, CO).

The live feature pipeline retrieves current readings from the AQICN API every hour. For historical data, a backfill process was required to generate a training dataset. During backfill, it was found that the AQICN free tier returns only the current reading rather than true historical observations, which produced a dataset with a constant, unusable AQI. This limitation was resolved by adopting the Open-Meteo Air Quality and Archive APIs for historical pollutant and weather data. The AQI value was then computed from the PM2.5 concentration using the United States Environmental Protection Agency (US EPA) breakpoint formula:

$$AQI = \frac{I_{hi} - I_{lo}}{C_{hi} - C_{lo}}(C - C_{lo}) + I_{lo} \tag{1}$$

where C is the PM2.5 concentration, and (C_{lo}, C_{hi}) and (I_{lo}, I_{hi}) are the surrounding concentration and AQI breakpoints. This correction yielded a realistic dataset of 4,344 hourly records spanning approximately six months.

Engineered features include time-based attributes (hour, day of week, month, weekend indicator), lagged AQI values, rolling means and standard deviations, lagged meteorological drivers, and cyclical encodings of hour and month. The forecasting target is the AQI a fixed number of hours into the future, created by shifting the AQI column.

IV. Exploratory Data Analysis

Exploratory analysis confirmed the dataset was complete and varied, with AQI ranging from 15 to 172 (mean 90). Several patterns emerged: air quality was worst in the early morning hours and cleanest around midday, consistent with nighttime temperature inversions; the winter months were substantially more polluted than spring; and weekly variation was minimal. Correlation analysis identified PM2.5 as almost perfectly correlated with AQI, with wind speed and temperature showing negative correlations consistent with pollutant dispersion. These findings informed the choice of features for modeling.

V. Model Training and Evaluation

Three models were trained and compared, in line with the project requirement to experiment with scikit-learn models and an advanced deep-learning model:

- **Ridge Regression** – a regularized linear baseline.
- **Random Forest** – a non-linear tree ensemble.
- **LSTM** – a recurrent neural network implemented in PyTorch.

PyTorch was used for the LSTM rather than TensorFlow because the available TensorFlow build required a version of the protobuf library that was incompatible with the Hopsworks client. The project specification permits either framework for the advanced model, so this was a sound engineering decision.

Data was split chronologically, training on the earliest 80% and testing on the most recent 20%, to provide a realistic estimate of forecasting performance and to avoid information leakage from the future. Models were evaluated using RMSE, MAE, and R-squared. Table I reports the results for the 24-hour-ahead forecast.

TABLE I
Model Performance (24-Hour-Ahead Forecast)

Model	RMSE	MAE	R^2
Ridge Regression	20.31	13.76	0.103
Random Forest	19.30	14.01	0.190
LSTM	25.84	18.76	-0.448

Random Forest achieved the best performance and was selected as the production model. A naive persistence baseline (assuming future AQI equals current AQI) achieved an R-squared of only 0.033, which Random Forest clearly exceeds, confirming that the model learns genuine structure. Notably, the simpler Random Forest outperformed the more complex LSTM, which overfitted on the relatively small dataset; this illustrates that greater model complexity does not guarantee better results when data is limited.

VI. Forecast Horizon and Its Challenges

An important finding of this project is that forecasting AQI far into the future is inherently difficult, because longer-range air quality depends heavily on future meteorological conditions that are not knowable from present data. Performance was substantially weaker at a 72-hour horizon than at 24 hours. Consequently, the system was built around a 24-hour-ahead model, and the three-day forecast is produced by recursive multi-step prediction: each day's forecast is fed back as input to predict the following day. This satisfies the requirement to forecast the next three days while remaining transparent about the fact that accuracy decreases for later days as errors accumulate.

VII. Explainability

SHAP (SHapley Additive exPlanations) was applied to the Random Forest model to explain its predictions. The analysis identified PM2.5 as the dominant predictor, followed by the recent multi-day AQI trend and seasonal information. The directional analysis confirmed intuitive relationships: higher PM2.5 increases predicted AQI, while higher wind speed decreases it. These results demonstrate that the model relies on physically meaningful factors, supporting its trustworthiness and satisfying the requirement for feature-importance explanations.

VIII. Automation with CI/CD

The feature and training pipelines were automated using GitHub Actions, a free serverless CI/CD platform. Two scheduled workflows were created: the feature pipeline runs every hour to collect live data, and the training pipeline runs daily to retrain the model. Both also support manual triggering for testing. API credentials are stored as encrypted GitHub secrets and injected at runtime, so no secret appears in the source code. During testing, missing dependencies required by the Hopsworks client for writing data (Apache Arrow and Kafka libraries) were identified and resolved by installing the Hopsworks client with its full set of Python extras. Both workflows were verified to run successfully.

IX. Web Application and Deployment

The user-facing dashboard was built with Streamlit. It loads the trained model and recent features from Hopsworks, computes the recursive three-day forecast, and presents the current AQI, a colour-coded three-day forecast, an interactive trend-and-forecast chart, and an automated hazardous-AQI alert. The alert triggers when any forecast day exceeds the unhealthy threshold, advising sensitive groups to limit outdoor activity. The application was deployed publicly on Streamlit Community Cloud, with the Hopsworks credential supplied through Streamlit's encrypted secrets. Because it reads from the continuously updated Feature Store and Model Registry, the dashboard automatically reflects the latest data and model.

X. Limitations and Future Work

The project meets its objectives but has several honest limitations. First, the historical dataset spans approximately six months; a longer history (one to two years) would likely improve all models, particularly the data-hungry LSTM. Second, multi-day forecasting accuracy is fundamentally limited because future AQI depends on unknowable future weather; incorporating weather *forecast* data as input features is a promising direction for substantial improvement. Third, the recursive forecasting approach accumulates error across days. Finally, the system currently models a single city. Future work could expand the dataset, integrate forecast weather, support multiple cities, and explore more advanced sequence models with sufficient data.

XI. Conclusion

The Pearls AQI Predictor is a complete, automated, serverless machine learning system that forecasts Karachi's air quality for the next three days. It collects data automatically, stores features and models in a managed registry, retrains on a schedule, and serves predictions through a publicly accessible dashboard with health alerts. The project fulfills all core requirements: an end-to-end prediction system, a scalable automated pipeline, an interactive dashboard showing real-time and forecasted data, and thorough documentation. Beyond the implementation, the project demonstrates sound engineering judgment in diagnosing data-quality issues, resolving dependency conflicts, selecting models based on empirical evidence, and presenting results honestly, including the genuine limitations of air quality forecasting.